



Міністерство освіти і науки України  
Національний технічний університет України  
“Київський політехнічний інститут імені Ігоря Сікорського”  
Факультет інформатики та обчислювальної техніки  
Кафедра автоматики та управління в технічних системах

Лабораторна робота №8  
**Технології розроблення програмного забезпечення**  
*« Патерни проектування: Composite, Flyweight, Interpreter,  
Visitor.»*  
*«System activity monitor»*

Виконав  
студент групи ІА–32:  
Феклістов Д.С.

Київ 2025

## 1. Мета роботи

Вивчити призначення та структуру патернів проектування: Composite, Flyweight, Interpreter та Visitor.

Практично реалізувати патерн Visitor (Відвідувач) у системі моніторингу для відокремлення алгоритмів експорту даних (генерація звітів у форматах Text та JSON) від структури класів сенсорів.

## 2. Теоретичні відомості

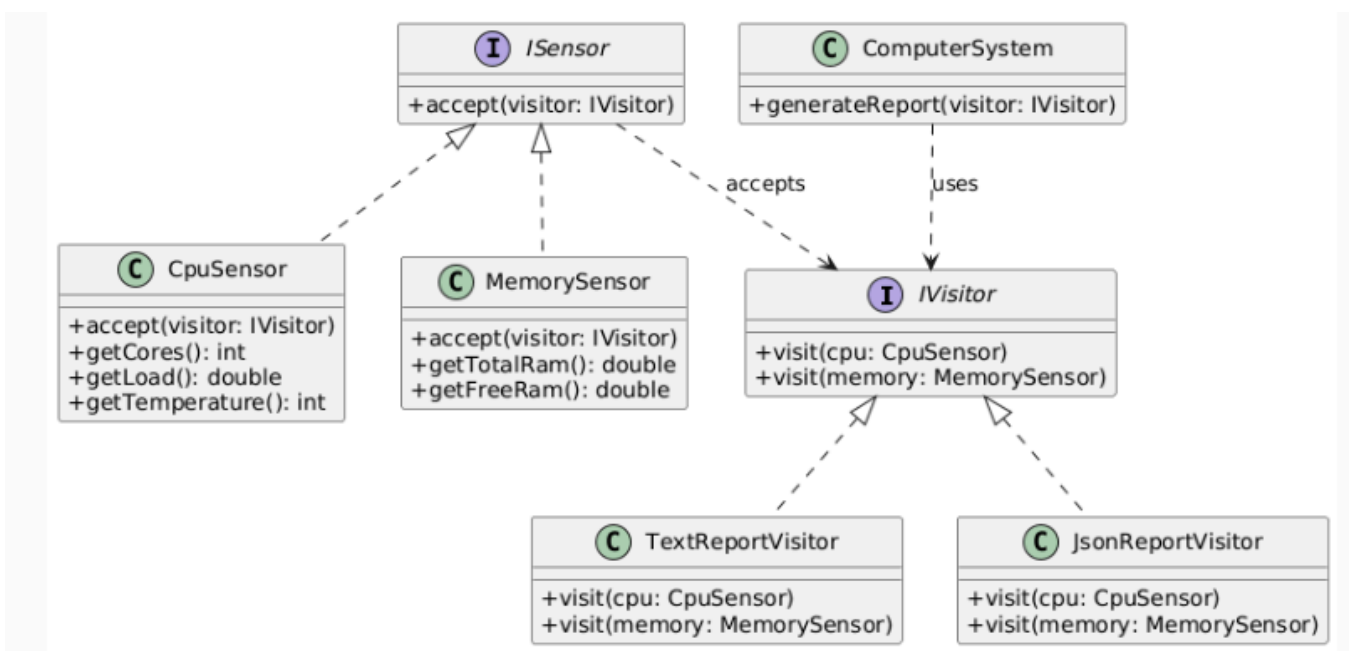
**Visitor (Відвідувач)** — це поведінковий патерн проектування, який дозволяє додавати нові операції до об'єктів без зміни їхніх класів.

**Суть патерну:** Замість того, щоб розміщувати логіку (наприклад, експорт у XML) всередині кожного класу даних, ми створюємо окремий клас "Відвідувач", який реалізує цю логіку. Об'єкти даних лише надають метод асерт, який дозволяє відвідувачу "зайти" і отримати доступ до даних.

**Переваги:**

- **Принцип відкритості/закритості (ОСР):** Можна додавати нові типи звітів (HTML, XML, CSV), не змінюючи код сенсорів.
- **Об'єднання логіки:** Код для генерації конкретного формату звіту зібраний в одному класі, а не розкиданий по всій програмі.

## 3. Діаграма класів (UML)



## 4. Хід роботи

1. **Створено інтерфейс ISensor:** Визначає метод accept(IVisitor visitor), який дозволяє відвідувачу обробляти об'єкт.
2. **Модифіковано класи сенсорів (CpuSensor, MemorySensor):**
  - Реалізовано інтерфейс ISensor.
  - Додано методи-геттери (getLoad, getFreeRam тощо) для надання доступу до реальних показників системи (використання OperatingSystemMXBean).
3. **Розроблено інтерфейс IVisitor:** Описує методи visit для кожного типу сенсора.
4. **Реалізовано конкретних відвідувачів:**
  - TextReportVisitor: Формує читабельний звіт для користувача у консолі.
  - JsonReportVisitor: Формує структурований JSON-рядок (наприклад, { "cpu": { "load": 12.5 } }), що зручно для передачі даних у веб-сервіси.
5. **Оновлено Фасад (ComputerSystem):** Додано метод generateReport, який приймає будь-якого відвідувача і передає його сенсорам.
6. **Тестування:** У класі Main виконано генерацію двох типів звітів на основі реальних даних системи.

## 5. Код програми

Файл: src/monitor/visitor/IVisitor.java

```
package monitor.visitor;

import monitor.sensors.CpuSensor;
import monitor.sensors.MemorySensor;

public interface IVisitor {

    void visit(CpuSensor cpu);

    void visit(MemorySensor memory);

}
```

### Файл: src/monitor/sensors/ISensor.java

```
package monitor.sensors;

import monitor.visitor.IVisitor;

public interface ISensor {
    // Цей метод впускає відвідувача
    void accept(IVisitor visitor);
}
```

### Фрагмент src/monitor/sensors/CpuSensor.java

```
package monitor.sensors;

import monitor.visitor.IVisitor;
import java.lang.management.ManagementFactory;
import java.lang.management.OperatingSystemMXBean;
import java.util.Random;

public class CpuSensor implements ISensor {

    //робимо біл полем класа
    private OperatingSystemMXBean osBean;

    public CpuSensor() {
        this.osBean =
ManagementFactory.getOperatingSystemMXBean();

        // "прогрів датчика
        // робимо одразу при запуску в холосту.
        if (osBean instanceof
com.sun.management.OperatingSystemMXBean) {
            ((com.sun.management.OperatingSystemMXBean)
osBean).getSystemCpuLoad();
        }
    }

    @Override
    public void accept(IVisitor visitor) {
```

```

        visitor.visit(this);
    }

    // Получение реальної завантаженості
    public double getLoad() {
        if (osBean instanceof
com.sun.management.OperatingSystemMXBean) {
            double load =
((com.sun.management.OperatingSystemMXBean)
osBean).getSystemCpuLoad();
            // якщо джава не вдалося зробити замір
повертаємо до 0.0
            return Double.isNaN(load) ? 0.0 : load * 100;
        }
        return 0.0;
    }

    // інші методи
    public int getCores() {
        return osBean.getAvailableProcessors();
    }

    public int getCurrentTemperature() {
        // Температуру імітуємо
        return new Random().nextInt(20) + 50;
    }

    public int getTemperature() {
        return getCurrentTemperature();
    }

    // метод для виводу
    public String getCpuInfo() {
        return String.format("CPU: Ядер - %d, Завантаження -
%.1f%%, Температура - %d°C",
            getCores(), getLoad(), getTemperature());
    }
}

```

Файл: src/monitor/visitor/JsonReportVisitor.java

```
package monitor.visitor;

import monitor.sensors.CpuSensor;
import monitor.sensors.MemorySensor;

public class JsonReportVisitor implements IVisitor {
    @Override
    public void visit(CpuSensor cpu) {
        System.out.println("{ \"cpu\": { \"load\": " +
cpu.getLoad() + ", \"temp\": " + cpu.getTemperature() + " }
}");
    }

    @Override
    public void visit(MemorySensor memory) {
        System.out.println("{ \"memory\": { \"free\": " +
memory.getFreeRam() + ", \"total\": " +
memory.getTotalRam() + " } }");
    }
}
```

## 6. Результати виконання

```
-> [LOG]: Ініціалізація файлової системи логів...
--- ЕТАП 1: Стандартна діагностика ---
[02:44:39] [ФАСАД] Запуск повної діагностики...
[02:44:39] CPU: Ядер - 12, Загрузка - 19,7%, Температура - 60°C
[02:44:39] RAM: 11,3 GB / 13,9 GB | HDD: 56,5 GB / 475,7 GB
[02:44:39] [ФАСАД] Діагностику завершено.

--- ЕТАП 2: Демонстрація VISITOR ---

[1] Формування текстового звіту:
[02:44:40] [VISITOR] Генерація звіту...
--- ЗВІТ ПО CPU ---
Ядер: 12
Навантаження: 9,9%
Температура: 56C
-----
--- ЗВІТ ПО RAM ---
Всього: 13,86 GB
Вільно: 2,56 GB
-----

[2] Експорт у JSON формат:
[02:44:40] [VISITOR] Генерація звіту...
{ "cpu": { "load": 9.928004859295303, "temp": 58 } }
{ "memory": { "free": 2.5617523193359375, "total": 13.855533599853516 } }
```

## 7. Висновок

В ході лабораторної роботи було реалізовано патерн **Visitor (Відвідувач)**. Це дозволило розширити функціонал системи моніторингу (додати можливість експорту даних у JSON та Text) без внесення змін до логіки отримання даних у класах сенсорів. Такий підхід значно спрощує додавання нових форматів звітів у майбутньому та робить код чистішим.

## 8. Відповіді на контрольні питання

1. Яке призначення шаблону «Компонувальник» (Composite)?  
Дозволяє згрупувати об'єкти у деревоподібну структуру і працювати з цією структурою так само, як і з одиничним об'єктом.
2. Яке призначення шаблону «Муха» (Flyweight)?  
Дозволяє економити оперативну пам'ять при роботі з великою кількістю об'єктів, виділяючи їх спільний стан (внутрішній) в окремі спільні об'єкти.
3. Яке призначення шаблону «Інтерпретатор» (Interpreter)?  
Визначає граматику для певної мови та механізм інтерпретації речень цієї мови (використовується, наприклад, для розбору SQL або математичних виразів).
4. Яке призначення шаблону «Відвідувач» (Visitor)?  
Дозволяє визначити нову операцію над об'єктами певної структури, не змінюючи класи цих об'єктів. Часто використовується для генерації звітів, експорту даних або валідації.